

Five Algorithmic Workflows for Offline-to-Online Personalized LLM Training

Technical Design Note

August 25, 2026

Abstract

This note turns five research ideas into executable algorithm specifications. Each section gives the supporting papers, the inherited method boundary, a step-by-step workflow, a mathematical objective, pseudocode, invariants, and rollback conditions. The common setting starts from an offline personalized policy and adapts a frozen-backbone language-model agent through shared LoRA parameters and user-conditioned embeddings. The five proposals address ownership-aware stability, stale-trajectory reuse, user-specific policy advantages, temporal negative weighting, and active preference acquisition. They are designs for implementation and evaluation, not reports of completed experiments.

Contents

1	Common Training Contract	3
1.1	Scope and evidence	3
1.2	Shared record and checkpoint schema	3
1.3	Shared lifecycle	4
1.4	Upper, middle, and lower algorithm layers	4
1.5	Common activation gate	4
2	Idea 1: Ownership-Cone Personalized DPO	4
2.1	Thesis and novelty boundary	4
2.2	Step-by-step workflow	4
2.3	Objective and projected update	5
2.4	Invariants, stopping, and tests	6
3	Idea 2: Support-Handoff Replay	7
3.1	Thesis and novelty boundary	7
3.2	Step-by-step workflow	7
3.3	Support test and losses	8
3.4	Invariants, stopping, and tests	8
4	Idea 3: Profile-Swap Policy Gradient	9
4.1	Thesis and novelty boundary	9
4.2	Step-by-step workflow	9
4.3	Profile-swap advantage	10
4.4	Invariants, stopping, and tests	11

5	Idea 4: Tomorrow’s Mistake	11
5.1	Thesis and novelty boundary	11
5.2	Step-by-step workflow	12
5.3	Temporal bilevel objective	13
5.4	Invariants, stopping, and tests	13
6	Idea 5: Checkpoint-Fork Querying	14
6.1	Thesis and novelty boundary	14
6.2	Step-by-step workflow	14
6.3	Virtual branches and fork score	15
6.4	Invariants, stopping, and tests	16
7	The Two Closest Per-User Training Methods	16
7.1	What counts as a close match	16
7.2	OPPU: one history, one adapter	17
7.3	SPRInG: one adapter through time	19
7.4	Concrete design for our policy	20

Table 1: Lifecycle used by all five workflows. The middle step changes by idea; the provenance and activation steps do not.

Phase	Required action	Output
Offline	Train the registered P-RLHF or AGENTIC-DPO parent on a chronological historical split.	Immutable checkpoint c_0 , fixed reference policy, optimizer receipt.
Collect	Run only the active checkpoint in the declared environment and validate every new record.	Versioned interaction ledger.
Adapt	Execute the idea-specific update without reading validation or future-test evidence.	Candidate c_{t+1} and complete update receipt.
Verify	Load c_{t+1} in a fresh worker and compare it with c_t on a fixed validation panel.	Activation or rollback decision.
Evaluate	Test activated checkpoints on untouched future periods from natural initial states.	Separate task-success and personalization estimates.

1 Common Training Contract

1.1 Scope and evidence

All five ideas begin with an offline personalized language-model policy and continue learning from chronological agent interactions. The common policy state is

$$\theta = (\phi, e_0, \{e_u : u \in \mathcal{U}\}), \quad (1)$$

where the backbone is frozen, ϕ is a shared LoRA adapter, e_0 is a generic soft prompt, and e_u is a private embedding for user u . This organization follows personalized RLHF and parameter-efficient fine-tuning [Li et al., 2024, Hu et al., 2022]. Preference losses use DPO when a pairwise label is available [Rafailov et al., 2023]; numerical updates use AdamW [Loshchilov and Hutter, 2019].

Evidence boundary. The cited papers establish prior algorithms and motivate the proposed transformations. They do not establish that any method in this note works. Every improvement claim therefore remains a hypothesis until a checkpoint-changing run and untouched future evaluation have been completed.

1.2 Shared record and checkpoint schema

Every trainable record must identify the user and profile version, task and environment, generating model, behavior checkpoint, rendered state, actions and tool calls, outcome or preference provenance, timestamp, data split, and training eligibility. Token log-probabilities and replayable state fingerprints are required when an algorithm uses them. Missing fields cause quarantine; they are never silently reconstructed.

Each candidate checkpoint belongs to an append-only directed acyclic graph. Its receipt contains the parent hash, adapter hash, optimizer-state hash, data-policy version, evaluation receipt, activation status, and rollback reason. A rejected candidate cannot generate later data or donate optimizer moments.

1.3 Shared lifecycle

1.4 Upper, middle, and lower algorithm layers

The *upper layer* names the optimized object: the shared LoRA and user-conditioned parameters in [equation \(1\)](#). The *middle layer* maps evidence to an update, such as preference optimization, replay, policy gradient, bilevel weighting, or active acquisition. The *lower layer* provides loops, buffers, projections, checkpointing, and stopping rules. A method is credited only for the layer it changes. Standard minibatching, AdamW, replay storage, and checkpoint hashing remain infrastructure.

1.5 Common activation gate

A candidate may become active only if all of the following hold:

1. its adapter and optimizer hashes load in a clean worker;
2. the update is finite and nonzero;
3. task success and safety are non-inferior on the fixed validation panel;
4. the declared personalization validation statistic improves;
5. no protected provenance or future-test boundary was crossed.

Failure restores both parent parameters and parent optimizer state. Runs stop when their fixed interaction, token, optimizer-step, GPU, or wall-clock budget is exhausted; after three consecutive windows without an activated candidate; or at a preregistered safety termination.

2 Idea 1: Ownership-Cone Personalized DPO

2.1 Thesis and novelty boundary

A shared parameter update can change behavior for every user, while an update to e_u affects only user u . OWNERSHIP-CONE P-DPO matches each stability constraint to this ownership boundary: other-user replay constrains the shared block, and same-user replay constrains the active private block.

Inherited: the P-RLHF parameterization and specific-plus-generic preference loss, DPO, LoRA, AdamW, replay, and gradient projection.

Proposed: project the *actual AdamW displacement* in two separate ownership-scoped cones. No global no-forgetting guarantee is claimed. The constraint controls only the first-order change of the sampled replay losses.

2.2 Step-by-step workflow

1. Train an immutable offline P-RLHF parent c_0 and freeze π_{ref} .
2. Collect preference pairs with the active checkpoint. Reject pairs without a comparator, user identity, behavior hash, and ranking provenance.
3. Insert eligible pairs into a replay buffer stratified by user, task, time, and policy version.
4. Draw a current minibatch for user u , balanced other-user anchors for shared parameters, and same-user anchors for e_u .
5. Advance AdamW moments functionally and form the candidate descent displacements d_s and d_p , including weight decay.

Table 2: Algorithm papers supporting Ownership-Cone P-DPO.

Supporting paper	Component used here	Missing component supplied by this proposal
P-RLHF [Li et al., 2024]	Shared LoRA, generic embedding, user embeddings, and personalized DPO.	It is offline and has no ownership-scoped online stability rule.
DPO, LoRA, AdamW [Rafailov et al., 2023, Hu et al., 2022, Loshchilov and Hutter, 2019]	Pairwise loss, parameter-efficient update space, and optimizer.	They do not decide which users may constrain which parameter block.
GEM and A-GEM [Lopez-Paz and Ranzato, 2017, Chaudhry et al., 2019]	Replay-gradient constraints and projection.	Their task constraints do not distinguish shared from user-private parameters or the displacement applied by AdamW.
CAGrad [Liu et al., 2021]	Gradient-conflict control across objectives.	It does not encode parameter ownership or chronological personalization.
FedPDPO [Zhu et al., 2026]	Shared and personalized preference optimization.	Its federated split does not provide the proposed deployed replay cones.

6. Project d_s through cross-user constraints and d_p through same-user constraints. Keep every $e_v, v \neq u$, frozen.
7. Apply a finite nonzero projected step and write its gradients, optimizer state, active constraints, and projection angles to the receipt.
8. Hash and load the candidate in a clean worker. Activate it only through the gate in [section 1.5](#); otherwise restore the parent.
9. Evaluate activated checkpoints on untouched future periods. These outcomes do not return to the reported training run.

2.3 Objective and projected update

For $z = (u, x, y^+, y^-)$ and conditioning embedding e , define

$$\Delta q_{\theta}(z, e) = \log \frac{\pi_{\theta}(y^+ | x, e)}{\pi_{\text{ref}}(y^+ | x, e)} - \log \frac{\pi_{\theta}(y^- | x, e)}{\pi_{\text{ref}}(y^- | x, e)}, \quad (2)$$

$$\ell_{\text{P-DPO}}(z; \theta) = -\log \sigma(\beta \Delta q_{\theta}(z, e_u)) - \lambda_g \log \sigma(\beta \Delta q_{\theta}(z, e_0)). \quad (3)$$

Let $\mathbf{s} = (\phi, e_0)$ and $\mathbf{p} = e_u$. AdamW proposes descent displacements d_s and d_p from the current minibatch. For cross-user shared anchors A_j^s and same-user private anchors A_k^p , set

$$a_j^s = \nabla_{\mathbf{s}} \ell_{\text{P-DPO}}(A_j^s; \theta), \quad a_k^p = \nabla_{\mathbf{p}} \ell_{\text{P-DPO}}(A_k^p; \theta). \quad (4)$$

The two projections are

$$d_s^* = \arg \min_d \frac{1}{2} \|d - d_s\|_2^2 \quad \text{s.t.} \quad (a_j^s)^\top d \geq -\epsilon_j^s / \eta \quad \forall j, \quad (5)$$

$$d_p^* = \arg \min_d \frac{1}{2} \|d - d_p\|_2^2 \quad \text{s.t.} \quad (a_k^p)^\top d \geq -\epsilon_k^p / \eta \quad \forall k. \quad (6)$$

Algorithm 1: Ownership-Cone P-DPO

Input: Offline parent c_0 ; replay buffer $\mathcal{R}$; validation panel V ; budgets B ; tolerances ϵ^s, ϵ^p

Output: Append-only sequence of activated checkpoints

```
1  $c \leftarrow c_0$ ; freeze backbone and  $\pi_{\text{ref}}$ 
2 while budget and patience remain do
3   collect eligible pairs with behavior checkpoint  $c$  and append to  $\mathcal{R}$ 
4   sample active user  $u$  and current minibatch  $D_u$ 
5   sample balanced cross-user anchors  $\mathcal{A}_s$  and same-user anchors  $\mathcal{A}_p$ 
6   for  $k = 1, \dots, K$  do
7     form actual AdamW displacements  $(d_s, d_p)$  on  $D_u$ 
8      $d_s^* \leftarrow \text{Project}(d_s, \mathcal{A}_s, \epsilon^s)$  using equation \(5\)
9      $d_p^* \leftarrow \text{Project}(d_p, \mathcal{A}_p, \epsilon^p)$  using equation \(6\)
10    update only  $(\phi, e_0, e_u)$ ; keep  $e_v$  fixed for  $v \neq u$ 
11    log optimizer moments, constraints, norms, and projection angles
12  save and hash candidate  $\hat{c}$  with parent  $c$ 
13  if  $\hat{c}$  loads cleanly and passes  $V$  then
14    | activate  $\hat{c}$ ;  $c \leftarrow \hat{c}$ 
15  else
16    | restore parameters and optimizer state of  $c$ 
```

Apply $\mathbf{s} \leftarrow \mathbf{s} - \eta d_s^*$ and $e_u \leftarrow e_u - \eta d_p^*$. Since $\ell(\mathbf{s} - \eta d) \approx \ell(\mathbf{s}) - \eta a^\top d$, each constraint bounds the sampled anchor’s first-order loss increase by ϵ . It says nothing about higher-order change, unobserved users, or future reward.

2.4 Invariants, stopping, and tests

- Other-user anchors never constrain another user’s private embedding. Same-user anchors never stand in for shared cross-user protection.
- A raw-gradient projection is a baseline, not an implementation of the proposal; the projected object must be the applied AdamW displacement.
- Every candidate has one parent, and failed candidates cannot seed data or optimizer state.
- Task success and personalization are gated and reported separately.

An adaptation window stops at its declared optimizer-step or token budget. The main attribution tests are unconstrained online P-DPO, one global cone, shared-only and private-only cones, ownership-swapped cones, and raw-gradient rather than AdamW-displacement projection. Equal initialization, pairs, optimizer settings, trainable parameters, tokens, and validation gates are required across these controls.

Evidence boundary. This section specifies a testable algorithm. It contains no measured result and does not imply that the cone is feasible, nonzero, or beneficial on the target stream.

Table 3: Algorithm papers supporting Support-Handoff Replay.

Supporting paper	Component used here	Missing component supplied by this proposal
P-RLHF [Li et al., 2024]	Offline personalized initialization and trainable parameter blocks.	No deployed trajectory-replay loop.
WebRL [Qi et al., 2025]	Online agent RL and replay of useful experience.	It does not preserve an expired trajectory solely as an executable state.
VCPO [Huang et al., 2026]	ESS as a diagnostic for stale off-policy updates.	It stabilizes weighted updates; it does not switch representation from action to state.
Balanced Replay [Lee et al., 2021]	Controlled mixing during offline-to-online transition.	It retains or balances samples rather than terminating their action eligibility.
Offline/online LLM RL [Lanchantin et al., 2025]	Evidence that synchronization and online data change LLM post-training.	It does not define reconstructible state continuation for personalized agents.

3 Idea 2: Support-Handoff Replay

3.1 Thesis and novelty boundary

A historical action may become unusable for off-policy learning even when the state it reached remains useful. SHR therefore gives each trajectory a monotone replay status:

`action_alive` $\longrightarrow$ `state_only` $\longrightarrow$ `quarantined`.

Supported trajectories contribute importance-weighted action gradients. Once support collapses, their actions expire permanently; a verified historical state may still seed a fresh current-policy continuation.

Inherited: P-RLHF initialization, off-policy weights, effective sample size (ESS), replay buffers, state resets, LoRA, and AdamW.

Proposed: the irreversible action-to-state handoff and its rule that expired action tokens can never contribute another gradient.

3.2 Step-by-step workflow

1. Train and hash an offline P-RLHF checkpoint.
2. Store each live trajectory twice: an action payload with behavior log-probabilities and a state payload with tool-boundary fingerprints.
3. Group action-alive records by behavior checkpoint and compatible reward semantics.
4. Recompute current-policy log-probabilities and normalized ESS for each sampled cohort.
5. If ESS remains above the fixed threshold, form a clipped, self-normalized action-replay loss.
6. On the first ESS failure, atomically mark the cohort `state_only`. Missing behavior log-probabilities cause the same transition.
7. For a state-only record, replay a sampled prefix in a clean sandbox. Continue only if its canonical fingerprint exactly matches the ledger.

8. Generate fresh suffixes with the active checkpoint and train only on suffix tokens using newly verified outcomes.
9. Mix the available action and state losses, execute AdamW, then apply the common candidate-load, activation, and rollback gate.

3.3 Support test and losses

For a behavior cohort C of size n , define clipped ratios

$$w_i = \exp[\text{clip}(\log \pi_{\theta}(\tau_i \mid u_i) - \log \mu_i(\tau_i \mid u_i), -\log 10, \log 10)] \quad (7)$$

and normalized ESS

$$\eta_C = \frac{(\sum_{i \in C} w_i)^2}{n \sum_{i \in C} w_i^2}. \quad (8)$$

Use action replay only when $n \geq 8$, $\eta_C \geq \kappa$ with $\kappa = 0.5$, and reward semantics remain compatible. With $\bar{w}_i = w_i / \sum_j w_j$ treated as stop-gradient and A_i a return centered within user-task strata,

$$\mathcal{L}_A = - \sum_{i \in C} \bar{w}_i A_i \sum_{j \in \tau_i} \log \pi_{\theta}(a_{ij} \mid h_{ij}, e_{u_i}). \quad (9)$$

For a state-only trajectory, sample a verified boundary s_{ik} , reconstruct it, and draw $G = 4$ fresh suffixes $\xi_{ig} \sim \pi_{\theta}(\cdot \mid s_{ik}, e_u)$. Using leave-one-out advantages $\tilde{A}_{ig}$ from newly verified rewards,

$$\mathcal{L}_S = - \frac{1}{G} \sum_{g=1}^G \tilde{A}_{ig} \sum_{j \in \xi_{ig}} \log \pi_{\theta}(a_{igj} \mid h_{igj}, e_u). \quad (10)$$

The update minimizes

$$\mathcal{L} = \frac{1}{2} \mathcal{L}_A + \frac{1}{2} \mathcal{L}_S + \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{parent}}), \quad (11)$$

renormalizing over the modes available in the minibatch. The clipped, self-normalized action estimator is biased by design.

3.4 Invariants, stopping, and tests

- Replay status is monotone. A state-only cohort never returns to action replay, even if a later policy raises its apparent ESS.
- An expired prefix receives no gradient. Only fresh suffix tokens can train the policy after handoff.
- State replay requires a clean-sandbox reconstruction and exact canonical fingerprint. Failure means quarantine, not approximate reuse.
- Evaluation always starts from natural task initial states, never from a privileged replay boundary.

An adaptation window stops after 200 optimizer steps, 500 fresh suffixes, or its token ceiling. The necessary controls are fresh-only learning, a fixed offline/online replay mixture, ESS-based discard, ESS learning-rate scaling, always-action replay, always-state replay, and a compute-matched sham handoff. All controls receive the same ledger, reward semantics, suffix budget, and optimizer implementation.

Evidence boundary. The state adapter, reconstruction contract, and learner remain to be implemented. Repository readiness or a valid fingerprint alone does not show that handoff improves either personalization or task success.

Algorithm 2: Support-Handoff Replay

Input: Parent c_0 ; dual-payload ledger $\mathcal{R}$; threshold κ ; suffix count G ; validation panel V

Output: Activated checkpoints and monotone replay statuses

```
1  $c \leftarrow c_0$ 
2 while budget and patience remain do
3   collect trajectories with  $c$ ; validate action and state payloads
4   sample behavior cohorts  $\{C\}$  from  $\mathcal{R}$ 
5    $\mathcal{L}_A \leftarrow 0$ ;  $\mathcal{L}_S \leftarrow 0$ 
6   foreach cohort  $C$  do
7     compute  $\eta_C$  by equation \(8\)
8     if  $C$  is action_alive,  $\eta_C \geq \kappa$ , and rewards are compatible then
9       | add the loss in equation \(9\) to  $\mathcal{L}_A$ 
10    else
11      | mark  $C$  state_only permanently
12      | sample a tool boundary and reconstruct it in a clean sandbox
13      | if the canonical fingerprint matches then
14        | generate  $G$  suffixes with  $c$  and add equation \(10\) to  $\mathcal{L}_S$ 
15      | else
16        | mark the record quarantined
17    take one clipped AdamW step on the available terms of equation \(11\)
18    save and hash candidate  $\hat{c}$ 
19    if  $\hat{c}$  loads and passes  $V$  then
20      | activate  $\hat{c}$ ;  $c \leftarrow \hat{c}$ 
21    else
22      | restore  $c$  and its optimizer state
```

4 Idea 3: Profile-Swap Policy Gradient

4.1 Thesis and novelty boundary

A trajectory can receive high personalized reward because it is simply good for everyone. PSPG holds the realized trajectories fixed and changes only the profile used for scoring. Its personalization advantage is the intended user’s within-group rank minus the mean rank under matched alternative profiles.

Inherited: the personalized parameterization, grouped policy gradient, frozen profile scorers, rank normalization, LoRA, and AdamW.

Proposed: a cross-profile rank contrast on the *same* trajectory group. It is an evaluative contrast, not a causal estimate of user identity.

4.2 Step-by-step workflow

1. Train an offline P-RLHF checkpoint and freeze the preference scorer version used by the next adaptation window.
2. For user-task context (u, x) , generate $G = 8$ trajectories with the exact active checkpoint.
3. Before observing scores, sample $K = 4$ alternative profiles matched on task feasibility, permissions, language, and available evidence.
4. Score the same eight trajectories under u and every alternative profile. Do not rerun the environment.

Table 4: Algorithm papers supporting Profile-Swap Policy Gradient.

Supporting paper	Component used here	Missing component supplied by this proposal
P-RLHF [Li et al., 2024]	Shared LoRA and user-conditioned embeddings learned from preferences.	It has no chronological on-policy policy-gradient loop.
DeepSeekMath/GRPO [Shao et al., 2024]	Group-relative advantages and clipped policy optimization.	Group normalization does not separate intended-user value from broadly useful behavior.
WebRL [Qi et al., 2025]	Online reinforcement learning for multi-turn language-model agents.	It has no persistent cross-profile personalization contrast.
Reward factorization [Shenfeld et al., 2025]	User-conditioned reward structure and low-dimensional preference variation.	It does not form an on-policy advantage by rescoring identical trajectories under other users.

5. Convert each profile’s scores into centered within-group ranks.
6. Subtract the alternative-profile mean rank from the intended-user rank. Separately standardize deterministic task success.
7. Combine the two advantages with a validation-selected coefficient and execute one clipped on-policy AdamW update.
8. Hash, load, validate, activate, or roll back the candidate.
9. Evaluate future task success and personalization separately.

4.3 Profile-swap advantage

Let $h_\omega(v, x, \tau_i)$ be a frozen score for profile v . For each $v \in \{u\} \cup C(u, x)$, rank the G trajectories from least to most preferred, use average ranks for ties, and define

$$q_{v,i} = \frac{2 \text{rank}_v(i) - G - 1}{G - 1}. \quad (12)$$

Thus $q_{v,i} \in [-1, 1]$ and $\sum_i q_{v,i} = 0$. The profile-swap advantage is

$$A_i^{\text{swap}} = q_{u,i} - \frac{1}{K} \sum_{v \in C(u,x)} q_{v,i}. \quad (13)$$

For deterministic task success s_i , define

$$A_i^{\text{task}} = \frac{s_i - \bar{s}}{\text{sd}(s_1, \dots, s_G) + 10^{-6}}, \quad (14)$$

and set it to zero when the group variance is zero. The training advantage is

$$A_i = \text{clip}(A_i^{\text{task}}, -2, 2) + \lambda \text{clip}(A_i^{\text{swap}}, -2, 2), \quad (15)$$

where λ is selected on offline validation and then frozen.

If μ is the behavior policy and $\rho_{it} = \pi_\theta(a_{it} \mid h_{it}, u) / \mu(a_{it} \mid h_{it}, u)$, minimize

$$\mathcal{L}_{\text{PSPG}} = -\mathbb{E}_{i,t}[\min\{\rho_{it} A_i, \text{clip}(\rho_{it}, 1 - \epsilon, 1 + \epsilon) A_i\}] + \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{parent}}). \quad (16)$$

Ranks make the update invariant to a strictly monotone recalibration applied separately to each profile scorer.

Algorithm 3: Profile-Swap Policy Gradient

Input: Parent c_0 ; frozen scorer h_ω ; panel sampler; G, K, λ ; validation panel V

Output: Activated personalized policy checkpoints

```
1  $c \leftarrow c_0$ 
2 while budget and patience remain do
3   sample eligible user-task context  $(u, x)$ 
4   generate  $\{\tau_i\}_{i=1}^G$  with active checkpoint  $c$ 
5   sample matched alternative panel  $C(u, x)$  before scoring
6   foreach  $v \in \{u\} \cup C(u, x)$  do
7     score the identical trajectories with  $h_\omega(v, x, \cdot)$ 
8     compute centered ranks  $q_{v,i}$  by equation \(12\)
9   compute  $A_i^{\text{swap}}, A_i^{\text{task}}$ , and  $A_i$ 
10  consume the rollout group once in a clipped update on equation \(16\)
11  update only  $(\phi, e_0, e_u)$  and log scorer, panel, policy, and optimizer versions
12  save and hash candidate  $\hat{c}$ 
13  if  $\hat{c}$  loads and passes  $V$  then
14    activate  $\hat{c}$ ;  $c \leftarrow \hat{c}$ 
15  else
16    restore  $c$  and its optimizer state
```

4.4 Invariants, stopping, and tests

- Every profile scores the same trajectory tokens; profile swapping never changes environment execution.
- Alternative profiles are sampled before scores are visible, and their identities and propensities are recorded.
- The scorer is frozen within an adaptation window and receives no policy-training gradient.
- Each rollout group is consumed for at most one optimizer epoch.
- Task success, safety, and permissions are not counterfactually swapped.

An adaptation window stops after 200 optimizer steps, 400 rollout groups, or its token ceiling. Required controls include raw-reward GRPO, intended-profile rank only, user-history normalization, global normalization, same-profile duplicate panels, randomly permuted identities, and a compute-matched extra scoring baseline. These tests distinguish identity contrast from rank normalization and additional scorer calls.

Evidence boundary. Cross-profile subtraction may remove generic quality, but it may also amplify scorer bias or penalize preferences shared by many users. Only matched future evaluation can decide whether the resulting policy is more personalized without losing task competence.

5 Idea 4: Tomorrow’s Mistake

5.1 Thesis and novelty boundary

AGENTIC-DPO contrasts an accepted action with a plausible same-state mistake. The hardest current mistake, however, may teach little about the user’s next tasks. TMNW weights current mistakes by a one-step estimate of forward transfer: would correcting this mistake reduce loss on the same user’s next already-observed interaction block?

Table 5: Algorithm papers supporting Tomorrow’s Mistake.

Supporting paper	Component used here	Missing component supplied by this proposal
Agentic-DPO [Chen and Yuille, 2026]	Same-state action preferences, current-policy negatives, SFT anchoring, and schema-preserving views.	Its hardest-negative rule is offline, non-personalized, and myopic.
P-RLHF [Li et al., 2024]	Shared and user-private parameterization.	It has no action-level negative refresh or temporal meta-weighting.
Example reweighting [Ren et al., 2018]	A virtual update that scores training examples by a later objective.	The query set is generic validation data rather than the same user’s next chronological interaction block.
Bilevel optimization [Franceschi et al., 2018]	Differentiation through an inner optimization step.	It does not specify agent mistakes, preference losses, or online user chronology.

Inherited: AGENTIC-DPO’s state-level preference loss, SFT anchor, schema preservation, and negative refresh; P-RLHF’s user-conditioned parameters; bilevel differentiation; and AdamW.

Proposed: chronological support/query blocks and a virtual optimizer step that assigns weight to each current negative according to next-block transfer. The one-step hypergradient is local and does not guarantee future reward.

5.2 Step-by-step workflow

1. Run the AGENTIC-DPO SFT warm-up, freeze its reference policy, and initialize the shared LoRA and user embeddings from offline history.
2. Partition each user’s eligible stream into immutable consecutive blocks $B(u, 1), B(u, 2), \dots$
3. Use $S = B(u, t - 1)$ as support and $Q = B(u, t)$ as the meta-query. Both must be observed before the update; Q is not validation or test data.
4. At every accepted state in S and Q , sample four one-step actions from the active checkpoint. Remove duplicates, schema violations, and ineligible negatives.
5. Assign transient logits to support negatives and take one differentiable functional AdamW step on their weighted loss.
6. Evaluate the inherited AGENTIC-DPO loss on Q through the virtual parameters, and differentiate it with respect to the support logits.
7. Convert the negative hypergradient into within-state weights. Delete the virtual parameters and moments.
8. From the original parent state, execute one real weighted AGENTIC-DPO AdamW step.
9. Hash, load, validate, activate, or roll back the candidate. If activated, move Q into the support role for the next block.

5.3 Temporal bilevel objective

At support state s_i , let a_i^+ be the accepted action and a_{ik}^- , $k = 1, \dots, K$, be fresh schema-valid negatives. Let $\ell_{\text{ADPO}}(i, k; \boldsymbol{\theta})$ denote the inherited anchored, action-level AGENTIC-DPO loss. Introduce transient logits $c_i \in \mathbb{R}^K$ and weights

$$w_{ik}(c_i) = \text{softmax}(c_i/\tau)_k. \quad (17)$$

The weighted support loss is

$$\mathcal{L}_S(\boldsymbol{\theta}, c) = \frac{1}{|S|} \sum_{i \in S} \sum_{k=1}^K w_{ik}(c_i) \ell_{\text{ADPO}}(i, k; \boldsymbol{\theta}). \quad (18)$$

Let the current AdamW state be $o = (m, v, r)$. A functional step computes

$$g(c) = \nabla_{\boldsymbol{\theta}} \mathcal{L}_S(\boldsymbol{\theta}, c), \quad (19)$$

$$m'(c) = \beta_1 m + (1 - \beta_1) g(c), \quad (20)$$

$$v'(c) = \beta_2 v + (1 - \beta_2) g(c)^2, \quad (21)$$

$$\tilde{\boldsymbol{\theta}}(c) = \boldsymbol{\theta} - \alpha \left[\frac{\hat{m}'(c)}{\sqrt{\hat{v}'(c)} + \varepsilon} + \lambda_{\text{wd}} \boldsymbol{\theta} \right]. \quad (22)$$

Using uniform query-negative weights, define

$$J(c) = \frac{1}{|Q|K} \sum_{q \in Q} \sum_{k=1}^K \ell_{\text{ADPO}}(q, k; \tilde{\boldsymbol{\theta}}(c)). \quad (23)$$

Starting from $c = 0$, take one meta step,

$$c^* = -\eta_{\text{meta}} \nabla_c J(c)|_{c=0}, \quad w_{ik}^* = \text{softmax}(c_i^*/\tau)_k. \quad (24)$$

Then discard $\tilde{\boldsymbol{\theta}}, m', v'$, stop gradients through w^* , and take one real AdamW step on $\mathcal{L}_S(\boldsymbol{\theta}, w^*)$ from the unmodified parent state.

5.4 Invariants, stopping, and tests

- Support and meta-query blocks belong to the same authenticated user and adjacent ordered periods. Future blocks cannot be backfilled.
- Virtual checkpoints are ephemeral, bitwise isolated, and never activatable.
- The query block is training evidence already observed before the update. It is disjoint from checkpoint-selection validation and untouched future evaluation.
- Every negative records its generating checkpoint and is consumed in one real update window.
- A rejected real candidate supplies neither future rollouts nor AdamW moments.

Skip a user update when either block has fewer than 16 eligible action states or no valid negative. Stop a window after 150 real optimizer steps, 300 negative-generation batches, or its token ceiling. Controls must include unmodified AGENTIC-DPO, personalized hardest-negative AGENTIC-DPO, uniform negative weights, generic validation reweighting, temporally shuffled same-user queries, other-user queries, and a sham method that pays for the virtual pass but applies the parent rule.

Evidence boundary. A low next-block loss after one virtual step is only a local compatibility signal. The experiment must test whether that signal predicts future personalization and task success, and whether its extra differentiation cost is justified.

Algorithm 4: Tomorrow’s Mistake temporal meta-weighting

Input: Parent c_0 ; ordered user blocks; negatives per state K ; meta rate η_{meta} ; validation panel V

Output: Activated checkpoints and negative-weight receipts

```
1  $c \leftarrow c_0$ 
2 while budget and patience remain do
3   choose user  $u$  with adjacent eligible blocks  $S = B(u, t - 1)$  and  $Q = B(u, t)$ 
4   if  $S$  or  $Q$  lacks the minimum eligible states then
5      $\perp$  buffer evidence and continue
6   generate  $K$  valid same-state negatives in  $S$  and  $Q$  with checkpoint  $c$ 
7   initialize support logits  $c_i \leftarrow 0$ 
8   clone adapter and AdamW state functionally
9   compute virtual parameters  $\tilde{\theta}(c_i)$  by equation (22)
10  evaluate  $J(c)$  on  $Q$  and compute  $c^*$  by equation (24)
11  delete every virtual parameter and moment; restore the bitwise parent state
12  stop gradients through  $w^*$  and take one real weighted update on  $S$ 
13  log block, negative-generator, hypergradient, optimizer, and policy hashes
14  save and hash candidate  $\hat{c}$ 
15  if  $\hat{c}$  loads and passes  $V$  then
16    activate  $\hat{c}$ ;  $c \leftarrow \hat{c}$ ; advance the block pair
17  else
18     $\perp$  restore  $c$  and its optimizer state
```

6 Idea 5: Checkpoint-Fork Querying

6.1 Thesis and novelty boundary

An uncertain label is not necessarily useful: either answer may induce nearly the same policy. CFQ asks for feedback only when the two possible labels produce virtual one-step checkpoints with different behavior on user-relevant probe states.

Inherited: P-RLHF and P-DPO, active preference collection, AdamW, LoRA, Jensen–Shannon divergence, and information-gain acquisition.

Proposed: score a query in *behavior space* after both exact label-conditioned optimizer branches, then commit only the observed branch. The finite probe score is not a guarantee of downstream utility.

6.2 Step-by-step workflow

1. Train and load an immutable offline P-RLHF parent.
2. In each user window, generate a fixed pool of same-task live trajectory pairs with the exact active checkpoint.
3. Keep only pairs matched on deterministic task success and safety, so the requested label targets personalization rather than obvious correctness.
4. Before scoring the pool, freeze one anchor minibatch, user-relevant probe states, and schema-valid action slates.
5. For each candidate and each possible label, clone the parent adapter and AdamW state functionally and execute the exact one-step P-DPO update.

Table 6: Algorithm papers supporting Checkpoint-Fork Querying.

Supporting paper	Component used here	Missing component supplied by this proposal
P-RLHF [Li et al., 2024]	Personalized DPO loss and shared/private parameterization.	It trains a fixed preference set and never allocates live queries.
SELM [Zhang et al., 2024]	Active exploration for online language-model alignment.	It does not value both labels by the behavior of their resulting checkpoints.
Hybrid Preference Optimization [Bose et al., 2024]	Joint use of offline preference data and online queries.	It does not define a decision-focused per-query acquisition score.
SPRInG [Kim and Kim, 2026]	Selective updates under continual personalization.	Likelihood novelty does not test whether two labels change future actions.
Information-gain acquisition [MacKay, 1992]	Select data by expected reduction of decision uncertainty.	Classical information gain does not include a functional LLM optimizer branch or agent action probes.

6. Evaluate both virtual checkpoints on every fixed action slate.
7. Compute the label-prior-weighted Jensen–Shannon divergence between their action distributions and choose the largest fork.
8. Abstain if the maximum score is below the calibrated threshold. Otherwise request one label and execute only its branch as a real update from the unchanged parent.
9. Hash, load, validate, activate, or roll back the real candidate. All virtual branches and unqueried labels expire.

6.3 Virtual branches and fork score

For candidate pair $q = (u, x, y^A, y^B)$, define

$$d_{\theta, e}(q) = \log \frac{\pi_{\theta}(y^A | x, e)}{\pi_{\text{ref}}(y^A | x, e)} - \log \frac{\pi_{\theta}(y^B | x, e)}{\pi_{\text{ref}}(y^B | x, e)}. \quad (25)$$

For hypothetical label $z \in \{+1, -1\}$, where $+1$ prefers y^A , use

$$\ell_z(q; \theta) = -\log \sigma(\beta z d_{\theta, e_u}(q)) - \lambda_g \log \sigma(\beta z d_{\theta, e_0}(q)). \quad (26)$$

Freeze a seven-pair eligible anchor minibatch H_t for the whole acquisition window and set

$$\mathcal{L}_{q, z} = \frac{1}{8} \left[\ell_z(q; \theta) + \sum_{h \in H_t} \ell(h; \theta) \right]. \quad (27)$$

From the same parent parameters and AdamW state (m, v, r) , each hypothetical branch computes

$$g_{q,z} = \nabla_{\theta} \mathcal{L}_{q,z}, \quad (28)$$

$$m'_{q,z} = \beta_1 m + (1 - \beta_1) g_{q,z}, \quad (29)$$

$$v'_{q,z} = \beta_2 v + (1 - \beta_2) g_{q,z}^2, \quad (30)$$

$$\tilde{\theta}_{q,z} = \theta - \alpha \left[\frac{\hat{m}'_{q,z}}{\sqrt{\hat{v}'_{q,z} + \varepsilon}} + \lambda_{\text{wd}} \theta \right]. \quad (31)$$

For each of $R = 8$ training-side probe states s_r , fix a slate $A_r = \{a_{r1}, \dots, a_{rK}\}$ of $K = 4$ valid actions. The branch distribution is

$$P_{q,z,r}(j) = \text{softmax}_j \left(|a_{rj}|^{-1} \log \pi_{\tilde{\theta}_{q,z}}(a_{rj} \mid s_r, e_u) \right). \quad (32)$$

Calibrate the label prior $p_q = \sigma(d_{\theta, e_u}(q)/T_c)$ only on eligible historical labels, and let $M_{q,r} = p_q P_{q,+,r} + (1 - p_q) P_{q,-,r}$. The acquisition score is

$$F(q) = \frac{1}{R} \sum_{r=1}^R [p_q D_{\text{KL}}(P_{q,+,r} \parallel M_{q,r}) + (1 - p_q) D_{\text{KL}}(P_{q,-,r} \parallel M_{q,r})]. \quad (33)$$

This is the mutual information between the binary label and the finite action slate under the approximation above.

6.4 Invariants, stopping, and tests

- Both virtual labels start from the same parent parameters, moments, anchors, probes, learning rate, and weight decay.
- Virtual branches leave persistent parameters and moments bitwise unchanged and can never be activated.
- Labels remain hidden until a candidate has been selected and its query receipt written.
- Probe states come from eligible training-side history, never validation or untouched future tasks.
- Every acquisition baseline sees the same pool, hidden labels, feedback budget, and real-update implementation.

A user window stops after one query and real update, an abstention, or its rollout/token ceiling. Required acquisition baselines are random selection, label entropy, SPRInG-style novelty, expected gradient norm, expected model change, and a compute-matched sham that executes both branches and probes but selects randomly. Parameter-space fork distance and first-order logit approximations test whether full behavior-space simulation is necessary.

Evidence boundary. The method may spend substantial compute to avoid a single weak label, and its score depends on probe coverage and prior calibration. The primary empirical question is personalization per acquired label and per activated update, with task success, safety, and total GPU cost reported separately.

7 The Two Closest Per-User Training Methods

7.1 What counts as a close match

Our unit of personalization is one user. A deployment run owns that user’s adapter, optimizer state, memory, and checkpoint history. The initial policy is trained from the user’s thoughts, conversations,

Algorithm 5: Checkpoint-Fork Querying

Input: Parent c_0 ; candidate-pool size M ; anchors H_t ; probes and slates; threshold δ ; validation panel V

Output: Queried labels, abstentions, and activated checkpoints

```
1  $c \leftarrow c_0$ 
2 while label budget, compute budget, and patience remain do
3   collect  $M$  same-task live pairs with active checkpoint  $c$ 
4   filter to matched task-success and safety status
5   freeze  $H_t$ , probes, slates, and calibrated label priors
6   foreach candidate  $q$  do
7     foreach  $z \in \{-1, +1\}$  do
8       clone the same parent adapter and AdamW state functionally
9       compute virtual branch  $\tilde{\theta}_{q,z}$  by equation \(31\)
10      evaluate fixed action slates and discard the virtual state
11    compute  $F(q)$  by equation \(33\)
12   $q^* \leftarrow \arg \max_q F(q)$ 
13  if  $F(q^*) < \delta$  then
14    record an abstention; expire the pool
15  else
16    request label  $z^*$  only for  $q^*$ 
17    from the unchanged parent, take one real AdamW step on  $\mathcal{L}_{q^*, z^*}$ 
18    save and hash candidate  $\hat{c}$ 
19    if  $\hat{c}$  loads and passes  $V$  then
20      activate  $\hat{c}$ ;  $c \leftarrow \hat{c}$ 
21    else
22      restore  $c$  and its optimizer state
```

corrections, and task records. Later interactions update the same policy. A shared backbone may be trained separately, but the target user’s raw history does not train another user’s deployed branch.

Under this definition, the two closest papers are OPPU [Tan et al., 2024] and SPRInG [Kim and Kim, 2026]. OPPU is the closest offline initializer: it trains one adapter from one user’s history. SPRInG is the closest continual method: it carries that user’s adapter forward and updates it from chronological interaction batches. Together they bracket our offline-to-online workflow.

We exclude methods that train one policy across many users and personalize only through a profile or prompt. They do not produce the user-owned policy studied here.

7.2 OPPU: one history, one adapter

Model and data. OPPU freezes a task-adapted base model with parameters θ and assigns user u a private PEFT module $\Delta\theta_u$. The deployed model is

$$M_u = M_{\theta \oplus \Delta\theta_u}. \quad (34)$$

The user’s history $H_u^{<t}$ contains only records observed before the current query. A record is either a supervised pair (x_i, y_i) or a plain text sequence x_i . The first form covers prior requests and accepted responses. The second can hold notes, conversation text, or other user-authored material.

Workflow.

Table 7: Why OPPU and SPRInG are the closest matches.

Property	OPPU	SPRInG	Our setting
Policy owner	One PEFT module per user	One evolving LoRA adapter per user	One user-owned policy branch
Historical input	Paired behaviors or plain text	Ordered query-response records	Thoughts, conversations, task trajectories, and outcomes
Offline update	Cross-entropy or next-token training	Pretrained period-0 adapter	User-history initialization
Later update	None	Selective SFT on each new period	Preference or outcome-driven online update
Persistent output	Personal adapter	Adapter lineage and bounded memory	Adapter, optimizer state, memory, and audit receipt

1. Adapt a shared base model to the task using users outside the target cohort, then merge the task LoRA into the base.
2. Freeze the shared base.
3. Initialize one LoRA module $\Delta\theta_u$ for the target user.
4. Train it only on H_u . When retrieval is enabled, each example may retrieve only records that preceded that example.
5. Save $\Delta\theta_u$. At inference, load the frozen base and this user’s adapter, then answer the new query.

Optimization. For paired history, OPPU minimizes conditional cross-entropy:

$$\mathcal{L}_{\text{pair}}(\Delta\theta_u) = - \sum_{(x_i, y_i) \in H_u} \sum_{j=1}^{|y_i|} \log p_{\theta, \Delta\theta_u}(y_{i,j} \mid \varphi(x_i), y_{i,<j}). \quad (35)$$

When history contains plain text only, the target is the next token:

$$\mathcal{L}_{\text{text}}(\Delta\theta_u) = - \sum_{x_i \in H_u} \sum_{j=1}^{|x_i|} \log p_{\theta, \Delta\theta_u}(x_{i,j} \mid x_{i,<j}). \quad (36)$$

The paper uses rank-8 LoRA on the query and value projections, learning rate 10^{-5} , and two or three fixed epochs. It does not report an online update, replay rule, rollback rule, or optimizer-state lifecycle.

Inputs and outputs. Training consumes a frozen base model, one user identifier, and that user’s historical records. It outputs one loadable adapter. Inference consumes the adapter and a new query; optional retrieval or a generated profile may also enter the prompt. The paper evaluates Llama-2-7B on seven LaMP tasks using the 100 users with the longest histories.

What transfers. OPPU gives us the correct ownership boundary and a practical offline initializer. It does not learn from task outcomes, pairwise preferences, or live feedback. It also treats each record as text rather than as a replayable agent trajectory. Our offline stage can use its per-user adapter design, but our record schema must retain actions, tools, outcomes, timestamps, and the generating checkpoint.

Algorithm 6: OPPU for one user

Input: Frozen task model M_θ ; chronological user history H_u ; optional retriever R

Output: Personal adapter $\Delta\theta_u$

- 1 initialize $\Delta\theta_u$
 - 2 **foreach** record $h_i \in H_u$ *in chronological order* **do**
 - 3 construct its target from (x_i, y_i) or right-shifted text x_i
 - 4 if retrieval is enabled, retrieve only from $H_u^{<i}$
 - 5 accumulate the loss in [equation \(35\)](#) or [equation \(36\)](#)
 - 6 update only $\Delta\theta_u$ for a fixed number of epochs
 - 7 save $\Delta\theta_u$ with its user and history-version identifiers
-

7.3 SPRInG: one adapter through time

Model and data. SPRInG starts from the same ownership model. User u has a frozen backbone M_θ , an adapter $\phi_u^{(t)}$, and a bounded memory $B_u^{(t)}$. New interactions arrive in periods:

$$D_u^{(t)} = \{(q_i, a_i)\}_{i=1}^{n_t}. \quad (37)$$

Period 0 supplies the initial adapter. Each later period updates that same adapter. The paper uses text-generation records, not tool trajectories or explicit rewards.

Workflow.

1. Score every new (q, a) under the frozen base and the current user-adapted model.
2. Keep the top 30% by drift score.
3. Copy $\phi_u^{(t-1)}$ and fine-tune the copy on the selected records.
4. Merge selected records with the old memory, score them again under the updated adapter, and retain the hardest 50.
5. For a new query, retrieve two memory records. Use them only when their similarity passes a fixed gate.
6. Decode by interpolating adapter-only and retrieval-conditioned token probabilities.

Selection and update. Let

$$p_{\text{base}}(q, a) = P(a \mid q; M_\theta), \quad (38)$$

$$p_{\text{adap}}^{(t-1)}(q, a) = P(a \mid q; M_{\theta, \phi_u^{(t-1)}}). \quad (39)$$

SPRInG scores a record by

$$S_t(q, a) = -\log \frac{p_{\text{adap}}^{(t-1)}(q, a)}{p_{\text{base}}(q, a)} + \alpha \log p_{\text{base}}(q, a). \quad (40)$$

The likelihood ratio favors behavior not yet captured by the adapter. The base likelihood suppresses malformed or low-quality text. If τ_p is the period’s percentile threshold,

$$D_{\text{drift}}^{(t)} = \{(q, a) \in D_u^{(t)} : S_t(q, a) \geq \tau_p\}. \quad (41)$$

The adapter then takes supervised next-token updates on $D_{\text{drift}}^{(t)}$ only:

$$\phi_u^{(t)} = \text{SFT} \left(\phi_u^{(t-1)}, D_{\text{drift}}^{(t)} \right). \quad (42)$$

Algorithm 7: SPRInG for one user

Input: Frozen M_θ ; initial $\phi_u^{(0)}$; stream $\{D_u^{(t)}\}_{t \geq 1}$; percentile p ; budget $N_{\max}$
Output: Adapter lineage $\{\phi_u^{(t)}\}$ and memories $\{B_u^{(t)}\}$

```
1  $B_u^{(0)} \leftarrow \emptyset$ 
2 for  $t = 1, 2, \dots$  do
3   score  $D_u^{(t)}$  with equation \(40\)
4   retain  $D_{\text{drift}}^{(t)}$  with equation \(41\)
5    $\phi_u^{(t)} \leftarrow \text{SFT}(\phi_u^{(t-1)}, D_{\text{drift}}^{(t)})$ 
6    $C_u^{(t)} \leftarrow D_{\text{drift}}^{(t)} \cup B_u^{(t-1)}$ 
7   rescore  $C_u^{(t)}$  under  $\phi_u^{(t)}$ 
8   retain  $B_u^{(t)}$  with equation \(44\)
```

Residual memory and inference. After the update, SPRInG rescans

$$C_u^{(t)} = D_{\text{drift}}^{(t)} \cup B_u^{(t-1)} \quad (43)$$

and recomputes [equation \(40\)](#) with $\phi_u^{(t)}$. It keeps the $N_{\max}$ highest-scoring records:

$$B_u^{(t)} = \text{Top}_{N_{\max}} \{(q, a) \in C_u^{(t)}; S'_t(q, a)\}. \quad (44)$$

For a test query, the same adapted model produces an adapter-only distribution p_{par} and, when memory passes the relevance gate, a retrieval-conditioned distribution p_{ret} . The final distribution is

$$p_{\text{fin}} = \lambda p_{\text{ret}} + (1 - \lambda) p_{\text{par}}. \quad (45)$$

Memory affects inference only. It is not replayed into later optimizer steps.

Inputs and outputs. Training consumes one user’s initial adapter and a chronological stream of query–response batches. Each period outputs a new adapter and memory snapshot. Inference consumes the current adapter, current memory, and a new query, and outputs a response. The main experiment uses 100 LongLaMP users, five chronological periods, Gemma-3-4B-IT, rank-8 LoRA, AdamW, two epochs, and a 90/10 update/evaluation split within each period.

What transfers. SPRInG is the closest end-to-end prior because it preserves user ownership through time. Its main gap is the learning signal: difficult-to-predict text is treated as preference drift, and updates still maximize likelihood. Our agent receives richer evidence. A completed task can include tool calls, environment outcomes, user corrections, and explicit preferences. Those signals should determine eligibility and reward; likelihood novelty alone cannot tell whether an action helped the user.

7.4 Concrete design for our policy

The two papers suggest a clean baseline for our candidate online updates:

1. Serialize user u ’s thoughts, conversations, and successful task records into a chronological offline history.
2. Train a user-owned adapter with the OPPU objective. Save the adapter, optimizer state, data boundary, and base-model hash as checkpoint $c_u^{(0)}$.

3. Collect later tasks only with the active user checkpoint. Preserve states, actions, tools, outcomes, corrections, and behavior log-probabilities.
4. Use a SPRInG-style selection stage, but score evidence by task validity, preference information, and provenance rather than likelihood novelty alone.
5. Apply the chosen update from this note. Activate it only after the shared validation and rollback gate.

This baseline separates inheritance from novelty. OPPU supplies the offline per-user policy. SPRInG supplies the chronological adapter lineage. Our contribution begins where supervised text prediction ends: agent evidence, preference- or outcome-driven updates, auditable optimizer state, and safe activation.

References

- Avinandan Bose, Zhihan Xiong, Aadirupa Saha, Simon Shaolei Du, and Maryam Fazel. Hybrid preference optimization for alignment: Provably faster convergence rates by combining offline preferences with online exploration. *arXiv preprint arXiv:2412.10616*, 2024. URL <https://arxiv.org/abs/2412.10616>.
- Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1812.00420>.
- Yixiong Chen and Alan Yuille. Agentic-DPO: From imitation to agentic policy optimization on expert trajectories. *arXiv preprint arXiv:2607.10601*, 2026. URL <https://arxiv.org/abs/2607.10601>.
- Luca Franceschi, Paolo Frasconi, Saverio Salzo, Riccardo Grazi, and Massimiliano Pontil. Bilevel programming for hyperparameter optimization and meta-learning. In *International Conference on Machine Learning*, 2018. URL <https://proceedings.mlr.press/v80/franceschi18a.html>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- Luke J. Huang, Zhuoyang Zhang, Qinghao Hu, Shangdong Yang, and Song Han. Stable asynchrony: Variance-controlled off-policy RL for LLMs. *arXiv preprint arXiv:2602.17616*, 2026. URL <https://arxiv.org/abs/2602.17616>.
- Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM personalization via selective parametric adaptation and retrieval-interpolated generation. *arXiv preprint arXiv:2601.09974*, 2026. URL <https://arxiv.org/abs/2601.09974>.
- Jack Lanchantin, Angelica Chen, Janice Lan, Xian Li, Swarnadeep Saha, Tianlu Wang, Jing Xu, Ping Yu, Weizhe Yuan, Jason Weston, Sainbayar Sukhbaatar, and Ilia Kulikov. Bridging offline and online reinforcement learning for LLMs. *arXiv preprint arXiv:2506.21495*, 2025. URL <https://arxiv.org/abs/2506.21495>.
- Seunghyun Lee, Younggyo Seo, Kimin Lee, Pieter Abbeel, and Jinwoo Shin. Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble. In *Conference on Robot Learning*, 2021. URL <https://arxiv.org/abs/2107.00591>.
- Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Leqi Liu. Personalized language modeling from personalized human feedback. *arXiv preprint arXiv:2402.05133*, 2024. URL <https://arxiv.org/abs/2402.05133>.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL <https://arxiv.org/abs/2110.14048>.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://arxiv.org/abs/1706.08840>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.

- David J. C. MacKay. Information-based objective functions for active data selection. *Neural Computation*, 4(4):590–604, 1992. doi: 10.1162/neco.1992.4.4.590.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2411.02337>.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2305.18290>.
- Mengye Ren, Wenyuan Zeng, Bin Yang, and Raquel Urtasun. Learning to reweight examples for robust deep learning. In *International Conference on Machine Learning*, 2018. URL <https://arxiv.org/abs/1803.09050>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Idan Shenfeld, Felix Faltings, Pulkit Agrawal, and Aldo Pacchiano. Language model personalization via reward factorization. *arXiv preprint arXiv:2503.06358*, 2025. URL <https://arxiv.org/abs/2503.06358>.
- Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. doi: 10.18653/v1/2024.emnlp-main.372. URL <https://arxiv.org/abs/2402.04401>.
- Shenao Zhang, Donghan Yu, Hiteshi Sharma, Han Zhong, Zhihan Liu, Ziyi Yang, Shuohang Wang, Hany Hassan, and Zhaoran Wang. Self-exploring language models: Active preference elicitation for online alignment. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2405.19332>.
- Kewen Zhu, Liping Yi, Zhiming Zhao, Zhuang Qi, Han Yu, and Qinghua Hu. FedPDPO: Federated personalized direct preference optimization for large language model alignment. *arXiv preprint arXiv:2603.19741*, 2026. URL <https://arxiv.org/abs/2603.19741>.